Runnable interface by using different Cases :

Case 1 :

Runnable interface by using Anonymous inner class :

```
package com.ravi.runnable_cases;

public class Case1
{
    public static void main(String[] args)
    {
        Runnable r1 = new Runnable()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                IO.println("Current thread name is :"+name);
            }
        };

        Thread t1 = new Thread(r1);
        t1.start();

    }
}
```

Case 2 :

Anonymous Inner class without reference using Thread class constructor :

```
package com.ravi.runnable_cases;

public class Case1
{
    public static void main(String[] args)
    {
        Thread t1 = new Thread(new Runnable()
        {
            @Override
            public void run()
            {
                String name = Thread.currentThread().getName();
                IO.println("Thread Name is :"+name);
            }
        });
        t1.start();

    }
}
```

Case 3 :

* Runnable interface by using Lambda

```
package com.ravi.runnable_cases;

public class Case1
{
    public static void main(String[] args)
    {
        Runnable r1 = () -> IO.println(Thread.currentThread().getName());
        Thread t1 = new Thread(r1);
        t1.start();
    }
}
```

Case 4 :

Passing Lambda in the thread class constructor without reference

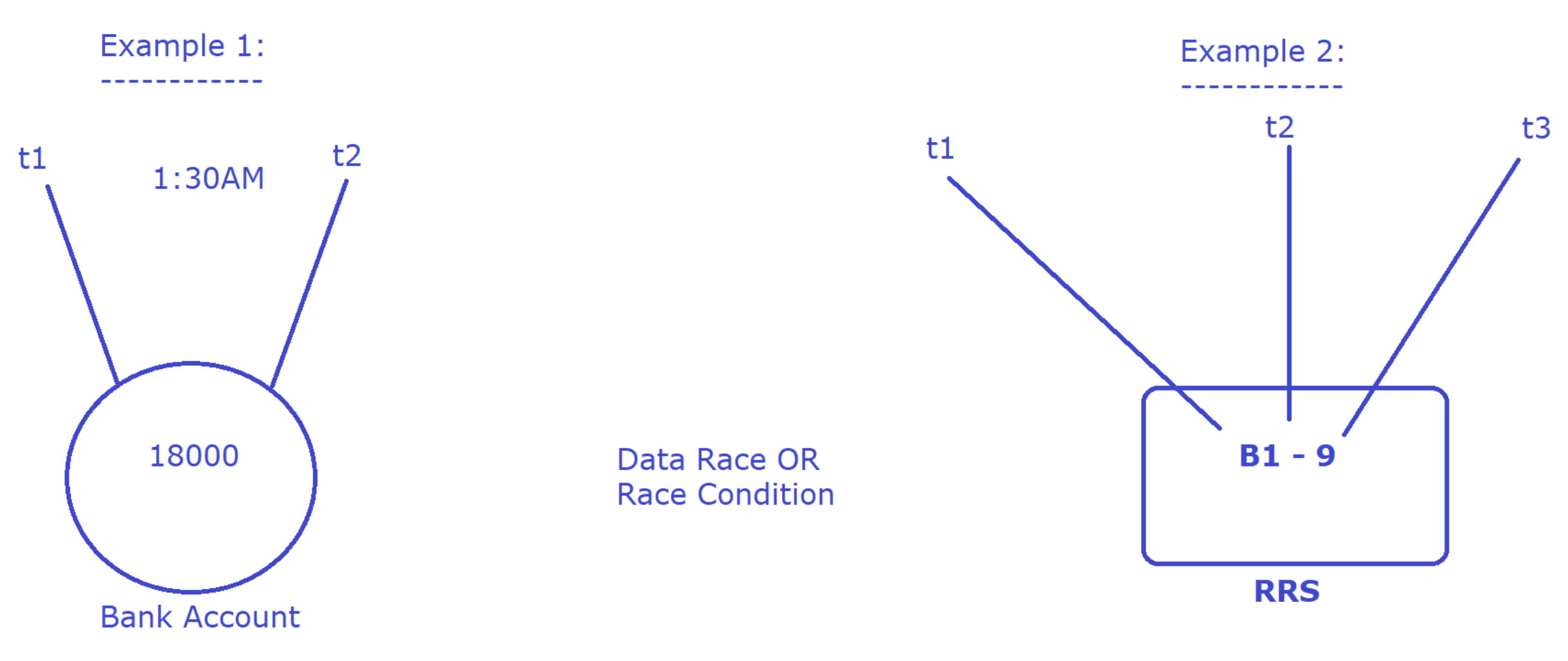
```
package com.ravi.runnable_cases;

public class Case1
{
    public static void main(String[] args)
    {
        new Thread(()->IO.println(Thread.currentThread().getName()), "Child1").start();

    }
}
```

Thread class Constructors:

Thread is a predefined class available in java.lang package. It contains 10 constructors. We have the list of constructors which we are using commonly.

1) Thread t1 = new Thread();
2) Thread t2 = new Thread(String name);
3) Thread t3 = new Thread(Runnable target);
4) Thread t4 = new Thread(Runnable target, String name);
5) Thread t5 = new Thread(ThreadGroup tg, String name);
6) Thread t6 = new Thread(ThreadGroup tg, Runnable target);
7) Thread t7 = new Thread(ThreadGroup tg, Runnable target, String name);

Drawback of Multithreading :



Multithreading is very good to complete our task as soon as possible but in some situation, It provides some wrong data or wrong result.

In Data Race or Race condition, all the threads try to access the resource at the same time so the result may be corrupted.

In multithreading if we want to perform read operation and data is not updatable data then multithreading is good but if the data is updatable data (modifiable data) then multithreading may produce some wrong result or wrong data which is known as Data Race OR race condition as shown in the diagram.

```
//Program :
-------------
package com.ravi.limitation_of_mt;

class Client implements Runnable
{
    private int availableSeat = 1;
    private int wantedSeat;

    public Client(int wantedSeat)
    {
        this.wantedSeat = wantedSeat;
    }

    @Override
    public void run()
    {
        String name = null;

        if(availableSeat >= wantedSeat)
        {
            name = Thread.currentThread().getName();
            IO.println(wantedSeat+" seat is allocated to "+name);
            availableSeat = availableSeat - wantedSeat;
        }
        else
        {
            name = Thread.currentThread().getName();
            System.err.println("Sorry "+name+" seat is not available");
        }



    }
}

public class RailwayReservation
{
    public static void main(String[] args) throws InterruptedException
    {
        Client client = new Client(1);

        Thread scott = new Thread(client,"Scott");
        Thread alen = new Thread(client,"Alen");

        scott.start();
        alen.start();
    }

}
```

Note : Here both the threads i.e scott and alen will get the seat so, not a good approach.

```
package com.ravi.limitation_of_mt;

//Lambda Approach

class Customer
{
    private double balance;

    public Customer(double balance)
    {
        this.balance = balance;
    }

    public void withdraw(double amount)
    {
        String name = null;

        if(amount <= this.balance)
        {
            name = Thread.currentThread().getName();
            IO.println(amount+" amount is withdraw by "+name);
            this.balance -= amount;
        }
        else
        {
            name = Thread.currentThread().getName();
            System.err.println("Sorry "+name+" insufficient Balance");
        }
    }
}

public class OnlineBankApplication {

    public static void main(String[] args)
    {
        Customer customer = new Customer(18000);

        Runnable r1 = ()->
        {
            customer.withdraw(18000);
        };

        Thread scott = new Thread(r1, "Scott");
        Thread alen = new Thread(r1, "Alen");

        scott.start();  alen.start();
    }

}
```

**** Synchronization :

* It is a technique through which we can control **multiple threads but accepting only one thread at a time for a single object.**

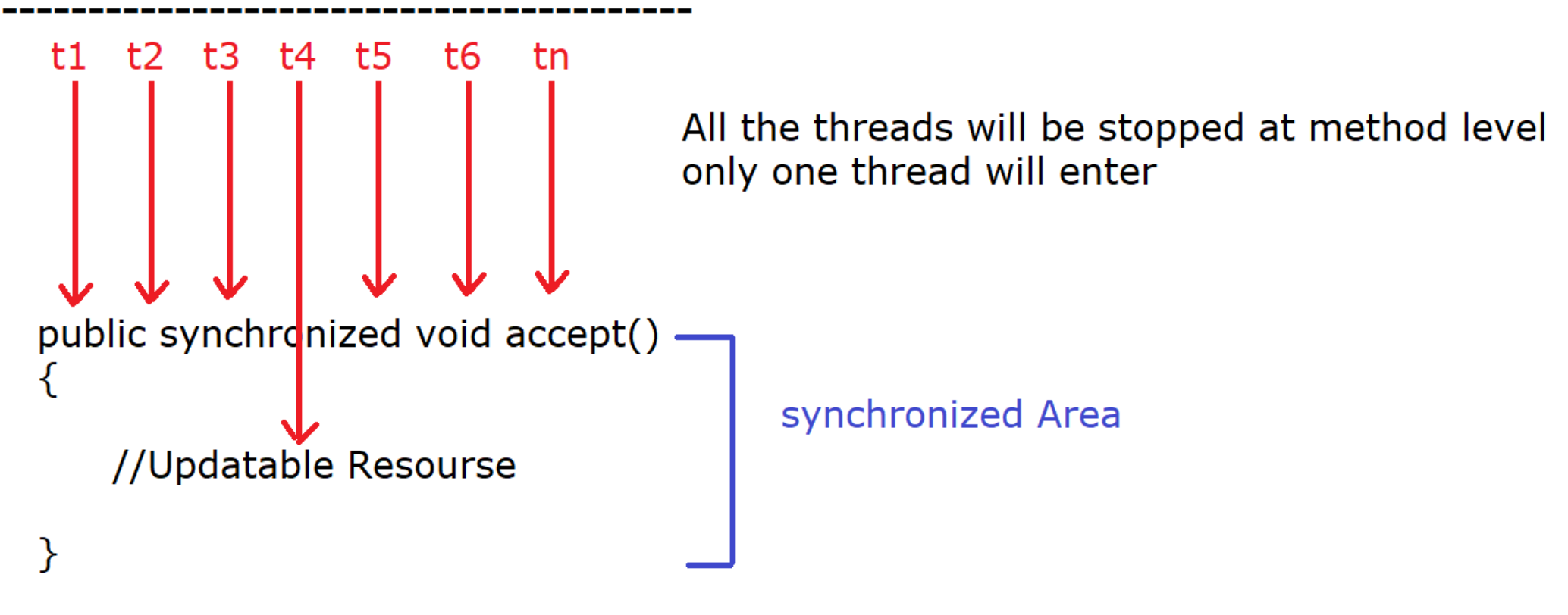
* In order to achieve synchronization we should use **synchronized keyword.**

* A synchronized area is a restricted area so, only one thread is allowed inside the synchronized area.

* Synchronization is divided into two types.

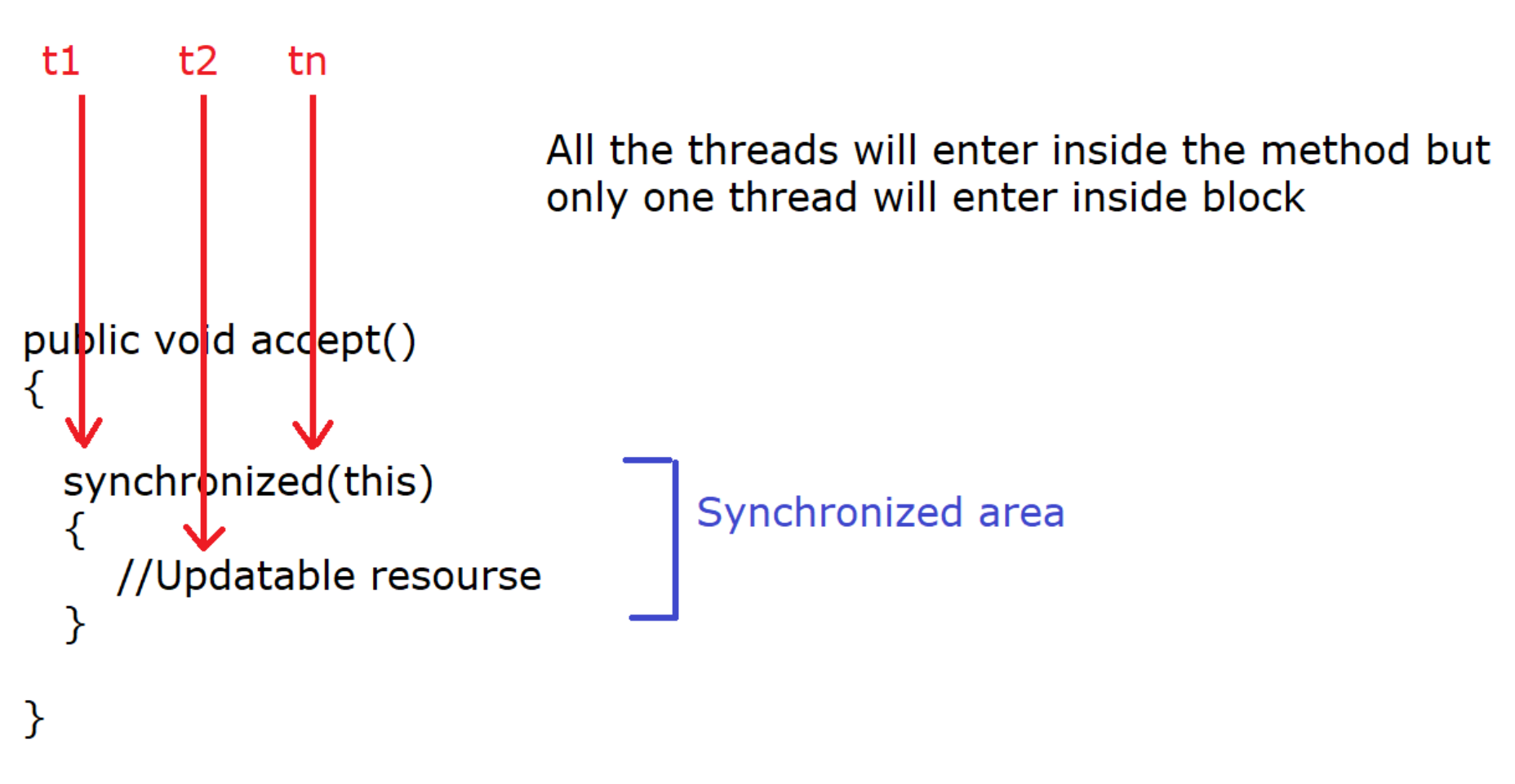
  1) Method Level Synchronization
  2) Block Level Synchronization

1) Method Level Synchronization :


t1 t2 t3 t4 t5 t6 tn
All the threads will be stopped at method level
only one thread will enter
public synchronized void accept()
{
//Updatable Resourse
}
synchronized Area

In method level synchronization, the entire method gets synchronized so all the threads will wait at method level and only one thread will enter inside the synchronized area as shown in the diagram.

2) Block Level Synchronization :


t1 t2 tn
All the threads will enter inside the method but
only one thread will enter inside block
public void accept()
{
synchronized(this)
{
//Updatable resourse
}
}
Synchronized area

2) Block level synchronization

In block level synchronization the entire method does not get synchronized, only the part of the method gets synchronized so all the threads will enter inside the method but only one thread will enter inside the synchronized block as shown in the diagram.

Note :- In between method level synchronization and block level synchronization, block level synchronization is more preferable because all the threads can enter inside the method so only the PART OF THE METHOD GETS synchronized so only one thread will enter inside the synchronized block.

Note : Synchronized area is a restricated area, with
permission only a thread can enter inside synchronized area.

How synchronization mechanism controls multiple threads OR Among 'n' number of threads which thread is allowed to enter inside synchronized area ?

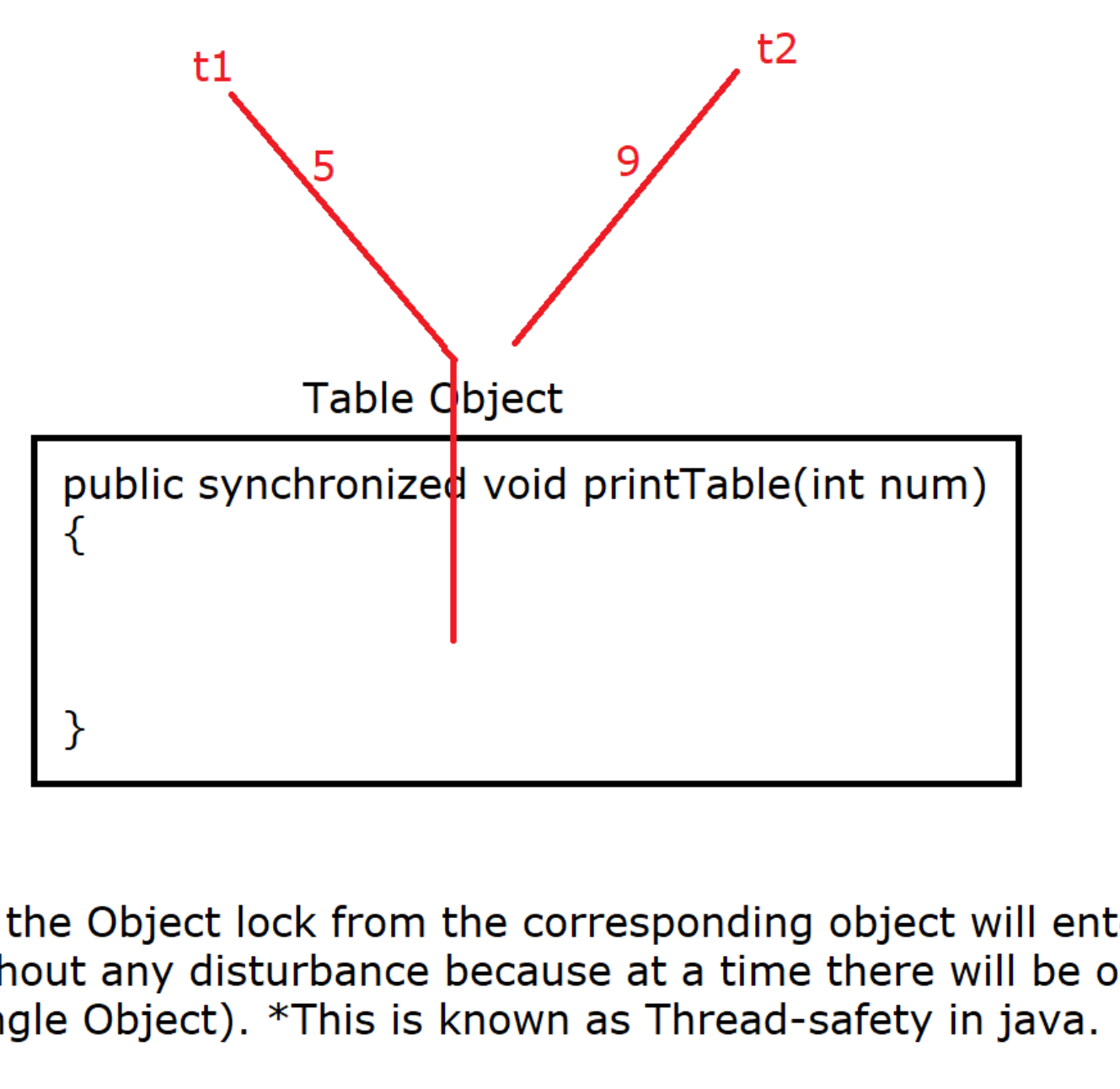
* In Java Environment, Every Object has a **lock OR monitor** which is provided by Object class at the time of creating the object.

  Example :

  Test t1 = new Test();

  One lock is associated with this object

* This lock OR monitor can be given to **only one thread** at a time.



The thread who acquires the Object lock from the corresponding object will enter inside the synchronized area, it will complete its task without any disturbance because at a time there will be only one thread inside the synchronized area(for single Object). *This is known as Thread-safety in java.

The thread which is inside the synchronized area, after completion of its task while going back will release the lock so the other threads (which are waiting outside for the lock (BLOCKED State)) will get a chance to enter inside the synchronized area by again taking the lock from the object and submitting it to the synchronization mechanism.

This is how synchronization mechanism controls multiple Threads.